

Universidad Nacional de Ingeniería
Facultad de Ingeniería Eléctrica y Electrónica

***SISTEMA DE RECONOCIMIENTO ÓPTICO DE
CARACTERES (OCR)***



Documentación Técnica del Proyecto
Informe integrador

Integrantes:

- Rosado Silva Manzur Arturo 20232230J
- Arteaga Choccare Fabrizzio Félix 20232114J
- Quino Sanchez Luis Eusebio 20230408F
- Quispe Guevara Vivian Ruth 20172143I

1. Resumen Ejecutivo

Este documento constituye la memoria técnica del proyecto “Sistema de Reconocimiento Óptico de Caracteres (OCR) para la Evaluación de Pruebas de Respuesta Corta”, desarrollado por un grupo de estudiantes de la Facultad de Ingeniería Eléctrica y Electrónica (FIEE) de la Universidad Nacional de Ingeniería (UNI). El proyecto nace de una pregunta incómoda pero real: en pleno proceso de digitalización educativa, ¿por qué un docente sigue dedicando hasta un 30% de su jornada a tareas mecánicas como calificar y transcribir notas a mano? La respuesta que encontramos fue que la brecha no está en la falta de tecnología, sino en la falta de soluciones que se adapten a la realidad del aula: papel físico, cámaras de celular imperfectas, poca iluminación y hojas mal alineadas.

Frente a este problema construimos un sistema que actúa como puente entre el examen físico tradicional y la gestión digital de calificaciones. El software recibe una fotografía del examen tomada con cualquier celular, la corrige automáticamente (perspectiva, iluminación y ruido), identifica al alumno, extrae sus respuestas mediante un motor OCR (Tesseract) apoyado en técnicas de visión por computadora (OpenCV), las compara contra un solucionario y genera de forma automática un archivo Excel con las notas finales, sin necesidad de que la institución invierta en tabletas, laptops o plataformas de examen 100% digitales.

Esta documentación recopila, en un solo lugar, todo el trabajo de análisis y desarrollo realizado hasta la fecha: el pitch del proyecto, la descomposición del trabajo (WBS), el análisis de procesos de negocio en su estado actual (AS-IS) y propuesto (TO-BE), la arquitectura de microservicios, el modelo de base de datos, la estructura del repositorio de código, la guía de despliegue en dos máquinas virtuales y la estrategia de integración continua con Jenkins. Los apartados que aún están en desarrollo por el equipo (modelado UML detallado y enlace final al repositorio de GitHub) se han dejado señalados explícitamente como pendientes, para ser completados en la siguiente actualización del informe.

2. Introducción y Planteamiento del Problema

La evaluación escrita en papel sigue siendo, hoy en día, el método predominante en gran parte de los colegios públicos, institutos y universidades de Latinoamérica. Si bien existen plataformas de evaluación 100% virtuales, su adopción masiva está limitada por la falta de equipos de cómputo suficientes para cada estudiante, la desigualdad de acceso a internet y la resistencia natural de instituciones que ya tienen procesos consolidados alrededor del papel.

Cuando se intenta digitalizar exámenes físicos con herramientas de reconocimiento de texto genéricas, los resultados suelen ser deficientes: una fotografía con sombras, una hoja ligeramente inclinada o una iluminación irregular del aula son suficientes para que el reconocimiento de caracteres falle. Ante este escenario, el docente termina abandonando la herramienta y regresa a la corrección manual, hoja por hoja, sumando y transcribiendo notas a un Excel, tal como se detalla en el diagrama de proceso actual (AS-IS) presentado en la sección 7.1.

El presente proyecto busca cerrar esta brecha diseñando un sistema robusto, capaz de tolerar las condiciones imperfectas de captura propias de un celular en un salón de clases real, sin exigir cambios en la forma de enseñar ni inversión en nuevo hardware para los estudiantes.

3. Objetivos del Proyecto

3.1 Objetivo General

Diseñar e implementar un sistema de reconocimiento óptico de caracteres (OCR) que automatice la calificación de pruebas de respuesta corta a partir de fotografías de exámenes físicos, reduciendo el tiempo y el margen de error de la corrección manual, y entregando al docente un reporte de notas listo para su uso.

3.2 Objetivos Específicos

- Desarrollar un módulo de preprocesamiento de imagen (corrección de perspectiva, binarización y limpieza de ruido) que compense las condiciones no controladas de captura con un celular.
- Integrar un motor OCR (Tesseract) para la lectura de las regiones de interés (ROI) correspondientes a la identificación del alumno y sus respuestas marcadas.
- Diseñar la lógica de comparación entre el texto extraído y el solucionario cargado por el docente, con asignación automática de puntaje.
- Construir una interfaz de usuario para la carga masiva de exámenes, la revisión visual de casos con baja confianza de lectura y la exportación de notas a Excel/CSV.
- Definir una arquitectura de microservicios desplegable en dos máquinas virtuales, orientada a separar el procesamiento pesado de imágenes del resto del sistema.
- Establecer un flujo de integración y despliegue continuo (CI/CD) que permita validar automáticamente los cambios de código antes de su publicación.

4. Equipo de Trabajo

El proyecto es desarrollado por un equipo de cuatro estudiantes de la FIEE-UNI, quienes asumen de forma conjunta las labores de análisis, diseño, desarrollo y despliegue. La elección de un enfoque de ingeniería “dura” (procesamiento de señales, matemática aplicada y algoritmos propios) frente al simple uso de herramientas genéricas de internet responde a la formación específica del equipo y busca dotar al sistema de mayor robustez ante los errores comunes del mundo real.

5. Alcance del Proyecto

Para delimitar con precisión qué funciones debía cubrir el sistema y cuáles quedaban fuera de este primer alcance, el equipo elaboró un diagrama de análisis funcional (técnica FAST — Function Analysis System Technique). Este diagrama organiza las funciones del sistema respondiendo a tres preguntas de control: hacia la izquierda se responde ¿POR QUÉ? se ejecuta una función (su propósito superior), hacia la derecha se responde ¿CÓMO? se logra (la función de la que depende) y en el eje vertical se ubican las funciones que ocurren ¿CUÁNDO? — es decir, de manera simultánea o continua durante toda la operación del sistema.

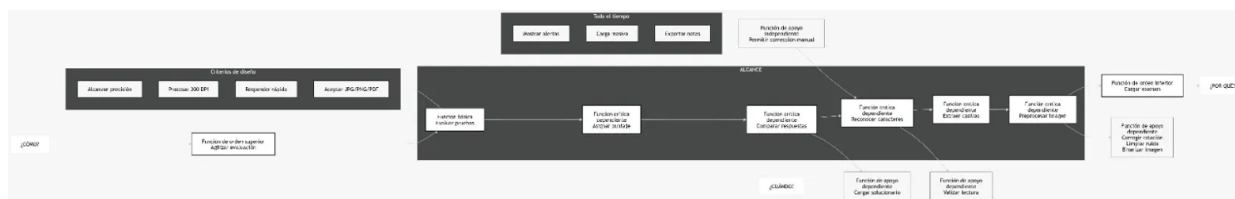


Figura 5.1. Diagrama de análisis funcional (FAST) del alcance del proyecto.

La función de orden superior identificada es “Cargar examen”, que da inicio a una cadena de funciones críticas dependientes entre sí: preprocesar imagen, extraer casillas, reconocer caracteres, comparar respuestas y asignar puntaje, hasta llegar a la función táctica principal del sistema: “Evaluar pruebas”. Esta cadena refleja exactamente el orden en que los módulos del sistema deben ejecutarse.

Adicionalmente, se identificaron funciones de apoyo, necesarias para que la cadena principal funcione correctamente pero que no forman parte de la secuencia crítica:

- Funciones de apoyo dependientes: cargar el solucionario o clave de respuestas, validar la lectura del OCR, corregir la rotación de la imagen, limpiar el ruido y binarizar la imagen.
- Función de apoyo independiente: permitir la corrección manual de una nota, disponible en cualquier momento del proceso, incluso si el reconocimiento automático fue exitoso.
- Funciones activas todo el tiempo: mostrar alertas de baja confianza de lectura, permitir la carga masiva de exámenes y exportar las notas finales.

Este análisis se complementó con cuatro criterios de diseño no funcionales que actúan como restricciones transversales a todo el sistema:

Criterio de diseño	Descripción
Alcanzar precisión	El sistema debe minimizar los errores de lectura y de calificación frente al solucionario.
Procesar a 200 DPI	Resolución mínima de trabajo para las imágenes de entrada, balance entre calidad de lectura y peso del archivo.
Responder rápido	El tiempo de procesamiento por examen debe permitir escanear un salón completo en pocos minutos.
Aceptar JPG/PNG/PDF	El sistema debe admitir los formatos de archivo más comunes generados por cámaras de celular y escáneres.

En conjunto, este diagrama fija el alcance funcional del proyecto: todo lo que está dentro de la cadena “cargar examen → evaluar pruebas” y sus funciones de apoyo forma parte del sistema; cualquier funcionalidad fuera de esta cadena (por ejemplo, la gestión curricular completa de un colegio, o la creación de exámenes en línea) queda fuera del alcance de esta primera versión.

6. Estructura de Descomposición del Trabajo (WBS)

Con el alcance funcional ya definido, el equipo tradujo el trabajo técnico en una Estructura de Descomposición del Trabajo (Work Breakdown Structure, WBS), organizada en cinco módulos o paquetes de trabajo principales. Esta estructura ha servido como hoja de ruta para la planificación de tareas y la asignación de responsabilidades dentro del equipo.

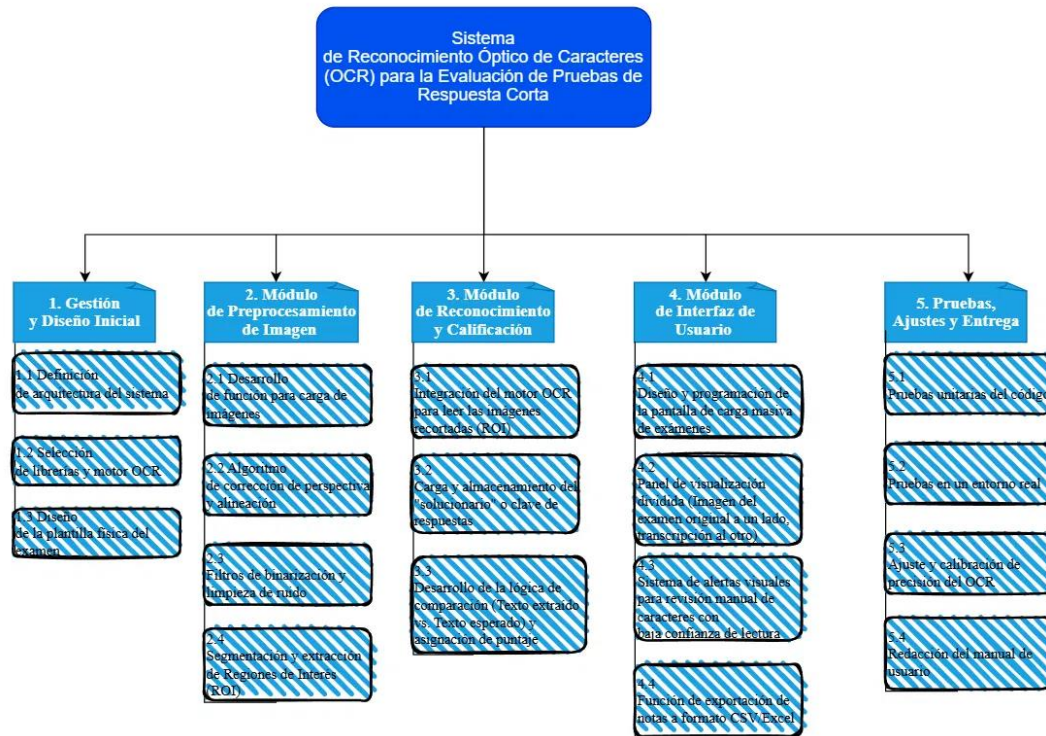


Figura 6.1. Estructura de Descomposición del Trabajo (WBS) del proyecto.

6.1 Descripción de los paquetes de trabajo

1. Gestión y Diseño Inicial

Paquete dedicado a las decisiones de arquitectura de más alto nivel: la definición de la arquitectura general del sistema, la selección de las librerías y el motor OCR a utilizar (Tesseract, apoyado en OpenCV) y el diseño de la plantilla física del examen, que estandariza dónde y cómo deben marcarse las respuestas para facilitar su posterior lectura automática.

2. Módulo de Preprocesamiento de Imagen

Agrupar el desarrollo de la función de carga de imágenes, el algoritmo de corrección de perspectiva y alineación (necesario porque las fotos tomadas con celular rara vez están perfectamente rectas), los filtros de binarización y limpieza de ruido, y finalmente la segmentación y extracción de las Regiones de Interés (ROI) donde se ubican el código del alumno y sus respuestas.

3. Módulo de Reconocimiento y Calificación

Comprende la integración del motor OCR sobre las imágenes recortadas (ROI), la carga y almacenamiento del solucionario o clave de respuestas, y el desarrollo de la lógica de comparación entre el texto extraído y el texto esperado, con la consecuente asignación automática de puntaje.

4. Módulo de Interfaz de Usuario

Incluye el diseño y la programación de la pantalla de carga masiva de exámenes, el panel de visualización dividida (que muestra la imagen original del examen junto a su transcripción para facilitar la verificación), el sistema de alertas visuales para revisión manual de caracteres con baja confianza de lectura, y la función de exportación de notas a formato CSV/Excel.

5. Pruebas, Ajustes y Entrega

Cierra el ciclo de desarrollo con las pruebas unitarias del código, las pruebas en un entorno real (con exámenes físicos reales, no solo datos simulados), el ajuste y calibración de la precisión del OCR, y la redacción del manual de usuario final. El presente documento amplía y formaliza justamente este último entregable.

7. Análisis de Procesos de Negocio (BPMN)

Para justificar de manera objetiva la necesidad del sistema, el equipo modeló el proceso de calificación de exámenes en dos versiones, siguiendo la notación BPMN (Business Process Model and Notation): el proceso actual (AS-IS), tal como lo realiza hoy el docente sin ninguna herramienta automatizada, y el proceso propuesto (TO-BE), una vez incorporado el sistema OCR. Comparar ambos diagramas permite visualizar con claridad en qué puntos exactos del flujo se elimina trabajo manual.

7.1 Proceso Actual (AS-IS): Calificación Manual

El diagrama AS-IS describe el proceso que actualmente ejecuta el docente, dividido en tres fases secuenciales: corrección, digitación y finalización.

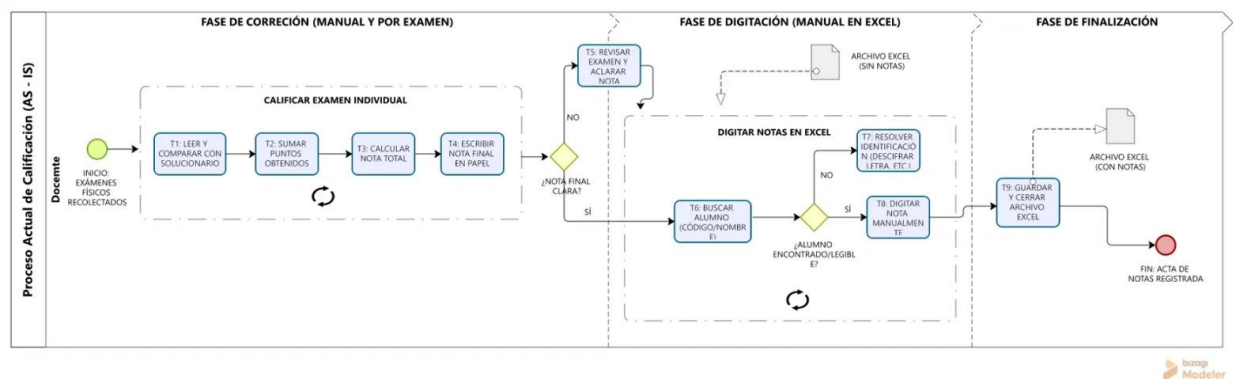


Figura 7.1. Proceso actual (AS-IS) de calificación de exámenes.

En la fase de corrección, por cada examen físico recolectado el docente debe leer y comparar las respuestas con el solucionario, sumar los puntos obtenidos, calcular la nota total y escribirla a mano sobre el papel; este ciclo se repite examen por examen. Si al finalizar la nota final no queda clara, el docente debe revisar nuevamente el examen para aclararla antes de continuar.

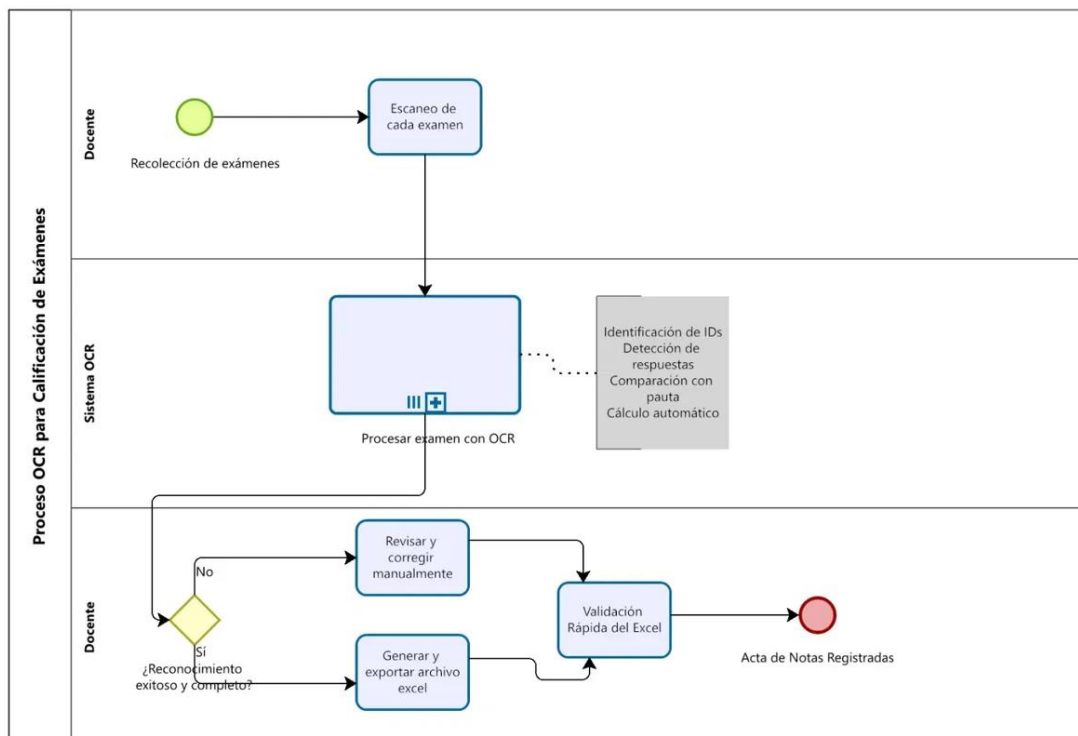
En la fase de digitación, ya con la nota definida en papel, el docente abre un archivo Excel vacío y debe buscar manualmente al alumno correspondiente por código o nombre. Si el alumno no es encontrado o su letra es ilegible, se activa un subproceso de resolución de identificación (por

ejemplo, describir la letra del alumno) antes de poder digitar la nota. Una vez identificado, la nota se digita manualmente en la celda correspondiente.

Finalmente, en la fase de finalización, el docente guarda y cierra el archivo Excel, generando el acta de notas registradas. Como puede observarse, el proceso completo depende enteramente de la atención sostenida del docente durante horas, con al menos dos ciclos de repetición (corrección y búsqueda de alumno) que son fuentes frecuentes de error humano y fatiga.

7.2 Proceso Propuesto (TO-BE): Calificación Asistida por OCR

El diagrama TO-BE incorpora al Sistema OCR como un nuevo carril (lane) dentro del proceso, ubicado entre el docente que recolecta los exámenes y el docente que valida el resultado final.



Powered by
bpmn.io
Modeler

Figura 7.2. Proceso propuesto (TO-BE) con el Sistema OCR integrado.

El flujo se inicia de la misma manera: el docente recolecta los exámenes físicos y los escanea (o fotografía) uno por uno. A partir de allí, la responsabilidad del procesamiento pasa al Sistema OCR, que ejecuta como una única tarea automatizada compuesta —representada en el diagrama como una subactividad con marcador de subproceso— las cuatro operaciones descritas en el WBS: identificación de códigos de alumno, detección de respuestas marcadas, comparación con la pauta o solucionario y cálculo automático de la nota.

El resultado de este procesamiento se somete a una única compuerta de decisión: ¿Reconocimiento exitoso y completo? Si la respuesta es negativa, el examen se enruta a una revisión y corrección manual puntual, acotada únicamente a los casos problemáticos y no a la totalidad de los exámenes. Si la respuesta es afirmativa, el sistema genera y exporta directamente el archivo Excel con las notas. Ambos caminos convergen en una validación rápida del Excel por parte del docente, quien solo debe confirmar que todo esté correcto antes de obtener la misma acta de notas registradas que en el proceso actual.

La comparación entre ambos modelos evidencia el principal beneficio del proyecto: el trabajo manual del docente deja de ser la regla y pasa a ser la excepción, reservada solo para los casos de baja confianza de lectura que el propio sistema señala.

8. Arquitectura del Sistema

La arquitectura del sistema se diseñó bajo un enfoque de microservicios distribuidos en dos máquinas virtuales (VM), separando explícitamente el procesamiento de imágenes —tarea intensiva en CPU— del resto de la lógica de negocio. Esta separación permite, por ejemplo, escalar o reemplazar el servidor de OCR sin afectar la disponibilidad de la API principal, y facilita que el servidor de OCR (VM2) pueda desplegarse incluso sin una dirección IP fija.

8.1 Vista General de la Arquitectura

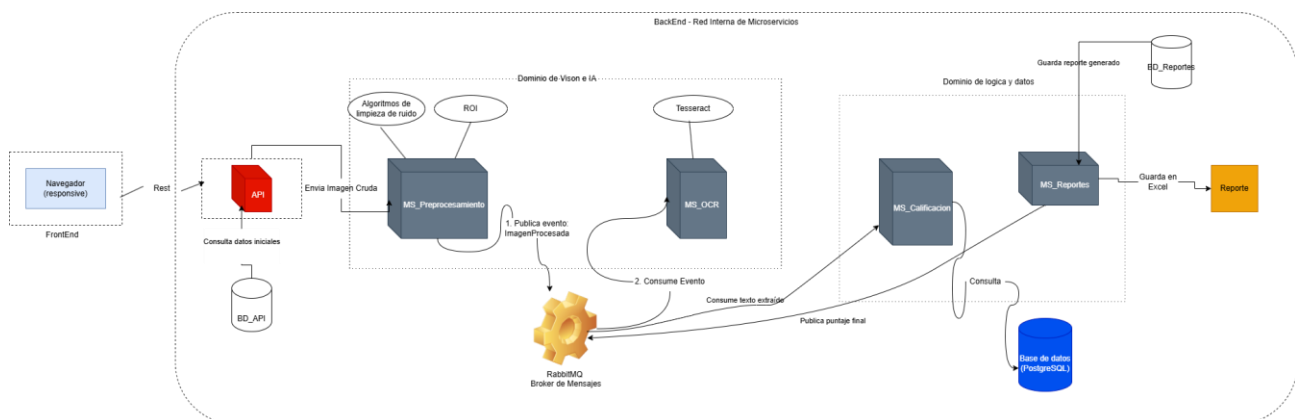


Figura 8.1. Vista general de la arquitectura de microservicios.

La VM1 actúa como servidor principal y aloja: el api_gateway (implementado en FastAPI), que expone los endpoints públicos del sistema; una instancia de RabbitMQ como middleware de mensajería (puerto 5672), que desacopla la comunicación entre servicios; el microservicio ms_calificacion, encargado de la lógica de negocio de calificación; y el microservicio ms_reporte, encargado de generar los reportes finales. Cada uno de estos servicios de negocio cuenta con su propia base de datos PostgreSQL independiente (bd_gateway, bd_calificacion y bd_reporte respectivamente), siguiendo el principio de “base de datos por servicio” propio de las arquitecturas de microservicios.

La VM2 actúa como servidor de OCR y no requiere IP fija, ya que solo necesita salida de red hacia la VM1 y no recibe conexiones entrantes directas. Aloja dos microservicios: ms_preprocesamiento, responsable de las tareas de visión por computadora (corrección de rotación, limpieza de ruido y

binarización descritas en el WBS), y ms_ocr, que ejecuta el motor Tesseract sobre las imágenes ya preprocesadas.

La comunicación entre ambas VMs se realiza exclusivamente a través del protocolo AMQP contra el RabbitMQ alojado en la VM1, lo cual evita acoplar directamente los microservicios entre sí: si la VM2 se cae temporalmente, los mensajes permanecen en cola en la VM1 hasta que el servicio de OCR vuelve a estar disponible.

8.2 Flujo de Comunicación Detallado

El siguiente diagrama de secuencia detalla, paso a paso, cómo viaja un examen desde que el docente lo sube hasta que se genera el reporte final de notas.

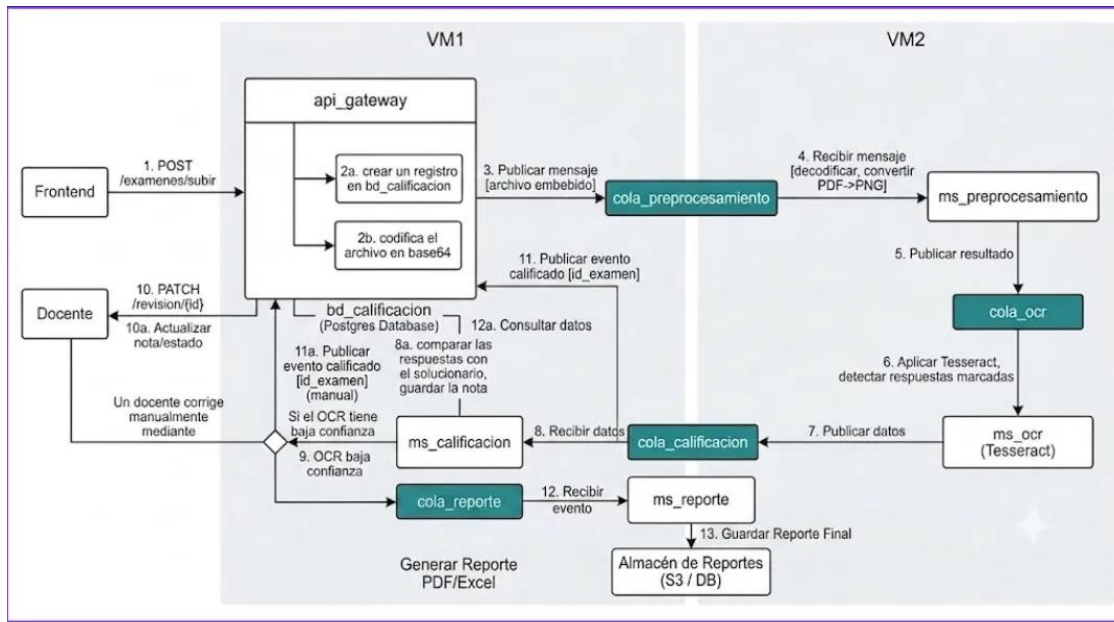


Figura 8.2. Flujo de comunicación entre componentes (VM1 y VM2).

El proceso se desencadena cuando el frontend realiza un POST a /exámenes/subir (paso 1) contra el api_gateway. Este último ejecuta dos acciones en paralelo: crea un registro del examen en bd_calificacion (paso 2a) y codifica el archivo recibido en base64 (paso 2b), para luego publicar un mensaje con el archivo embebido en la cola_preprocesamiento (paso 3).

Del lado de la VM2, el microservicio ms_preprocesamiento recibe el mensaje (paso 4), decodifica el archivo y, de ser necesario, lo convierte de PDF a PNG, aplicando además las correcciones de rotación y limpieza descritas anteriormente. El resultado se publica en la cola_ocr (paso 5), donde es recogido por ms_ocr (paso 6), que aplica Tesseract y detecta las respuestas marcadas por el alumno.

El resultado del reconocimiento se publica en la cola_calificacion (pasos 7 y 8), regresando así a la VM1. Aquí, ms_calificacion compara las respuestas reconocidas contra el solucionario cargado previamente y guarda la nota en bd_calificacion (paso 8a). Si el nivel de confianza del OCR es bajo, el sistema habilita al docente la posibilidad de corregir manualmente la nota mediante un PATCH a

/revision/{id} (pasos 9, 10 y 10a), cerrando el ciclo de excepción descrito en el proceso TO-BE (sección 7.2).

Finalmente, ms_calificacion publica un evento de examen calificado en la cola_reporte (pasos 11 y 11a), que es consumido por ms_reporte (paso 12). Este último consulta los datos definitivos en bd_calificacion (paso 12a) y genera el reporte final en formato PDF/Excel, almacenándolo en el repositorio de reportes (paso 13), ya sea un bucket S3 o una base de datos documental.

8.3 Modelo de Bases de Datos

Siguiendo el principio de independencia de datos entre microservicios, el sistema utiliza tres bases de datos PostgreSQL separadas, cada una de uso exclusivo de su microservicio correspondiente:

Base de datos	Usada por	Contenido
bd_gateway	api_gateway	Cursos, Solucionarios
bd_calificacion	ms_calificacion	Exámenes, Respuestas de alumnos
bd_reporte	ms_reporte	Reportes generados

Figura 8.3. Distribución de bases de datos por microservicio.

Base de datos	Usada por	Contenido
bd_gateway	api_gateway	Cursos, solucionarios
bd_calificacion	ms_calificacion	Exámenes, respuestas de alumnos
bd_reporte	ms_reporte	Reportes generados

Esta separación evita que un microservicio consulte o modifique directamente los datos de otro, forzando toda comunicación entre servicios a pasar por eventos publicados en RabbitMQ, lo cual reduce el acoplamiento y facilita que cada base de datos evolucione de forma independiente.

9. Estructura del Código Fuente

El código fuente del proyecto se organiza en dos carpetas independientes dentro del repositorio, una por cada máquina virtual, reflejando exactamente la separación de responsabilidades descrita en la arquitectura (sección 8).

9.1 Código de la VM1 (Servidor principal)

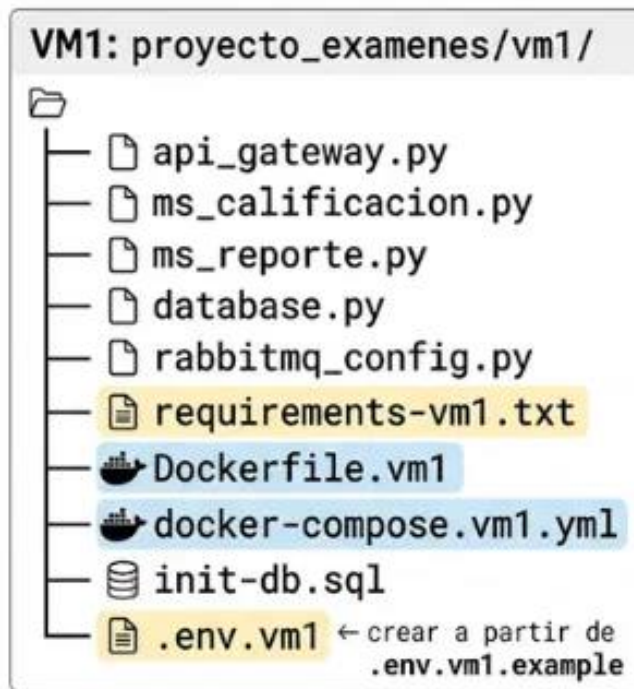


Figura 9.1. Estructura de carpetas de la VM1.

La carpeta `proyecto_examenes/vm1/` contiene los tres microservicios de negocio (`api_gateway.py`, `ms_calificacion.py` y `ms_reporte.py`), el módulo compartido `database.py` para la conexión a PostgreSQL y `rabbitmq_config.py` para la configuración del cliente de mensajería. A nivel de infraestructura incluye `requirements-vm1.txt` con las dependencias de Python, un `Dockerfile.vm1` y su respectivo `docker-compose.vm1.yml` para levantar los tres microservicios junto con sus bases de datos y RabbitMQ, el script `init-db.sql` para la inicialización de esquemas, y el archivo `.env.vm1` (creado a partir de la plantilla `.env.vm1.example`) con las variables de entorno sensibles, que no se versiona en el repositorio.

9.2 Código de la VM2 (Servidor de OCR)

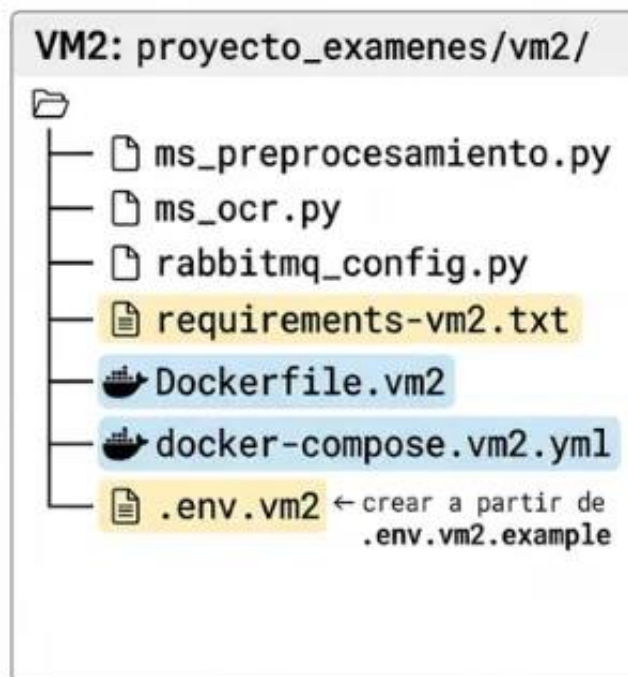


Figura 9.2. Estructura de carpetas de la VM2.

De manera análoga, `proyecto_examenes/vm2/` contiene los microservicios `ms_preprocesamiento.py` y `ms_ocr.py`, su propio `rabbitmq_config.py` para conectarse remotamente al RabbitMQ de la VM1, el archivo de dependencias `requirements-vm2.txt`, el `Dockerfile.vm2` y `docker-compose.vm2.yml`, y su archivo de entorno `.env.vm2` (a partir de `.env.vm2.example`) donde se configura, entre otros parámetros, la IP de la VM1 a la que debe conectarse.

9.3 Repositorio en GitHub

<https://github.com/Manzur68/SISTEMA-DE-RECONOCIMIENTO-PTICO-DE-CARACTERES-OCR->

10. Despliegue e Infraestructura

Esta sección funciona como manual operativo para levantar el sistema completo en un entorno de dos máquinas virtuales, cubriendo requisitos previos, configuración de red, secuencia de arranque y configuración del frontend.

10.1 Requisitos Previos en Ambas VMs

Ambas máquinas virtuales requieren Docker Engine y Docker Compose v2 instalados. La VM1 necesita, además, tener abiertos los puertos 8000 (API Gateway), 5672 (RabbitMQ/AMQP) y 15672 (panel de administración de RabbitMQ, opcional), así como un mínimo de 4 GB de RAM. La VM2 no requiere abrir puertos entrantes —solo sale hacia la VM1— pero sí necesita al menos 4 GB de RAM debido al consumo de CPU y memoria que implica ejecutar Tesseract.

Requisitos previos en ambas VMs {

Requisito	VM1	VM2
Docker Engine	Requiere	Requiere
Docker Compose v2	Requiere	Requiere
Puertos abiertos	8000, 5672, 15672	— (solo sale, no recibe)
RAM mínima	4 GB	4 GB (Tesseract consume CPU/RAM)
Conectividad	VM2 debe alcanzar el puerto 5672 de VM1	Debe alcanzar VM1:5672

Instalar Docker (Debian/Ubuntu/Kali) — ejecutar en AMBAS VMs

```
curl -fsSL https://get.docker.com | sudo sh sudo usermod -aG  
docker $USER newgrp docker
```

Abrir puertos necesarios (solo en VM1)

```
sudo ufw allow 8000/tcp      # API Gateway  
sudo ufw allow 5672/tcp     # RabbitMQ (AMQP) — VM2 debe poder llegar aquí  
sudo ufw allow 15672/tcp    # Panel de administración RabbitMQ (opcional)  
sudo ufw reload
```

Figura 10.1. Requisitos previos e instalación de Docker en ambas VMs.

La instalación de Docker se realiza en ambas VMs con el script oficial (`curl -fsSL https://get.docker.com | sudo sh`) seguido de la incorporación del usuario actual al grupo docker. En la VM1, adicionalmente, deben abrirse los tres puertos mencionados mediante el firewall (`ufw allow 8000/tcp`, `5672/tcp` y `15672/tcp`), ya que la VM2 debe poder alcanzar el puerto 5672 de la VM1 para publicar y consumir mensajes de RabbitMQ.

10.2 Configuración de Red entre VMs

Dado que la VM2 se comunica con la VM1 exclusivamente a través de RabbitMQ, la dirección IP de la VM1 debe configurarse manualmente en dos archivos distintos cada vez que dicha IP cambie (por ejemplo, al migrar de entorno de pruebas a producción).

Dónde se debe cambiar la IP

1 – Archivo `.env.vm2` (en VM2)

```
proyecto_examenes/vm2/.env.vm2
RABBITMQ_HOST=192.168.18.97 # ← CAMBIAR por la IP real de VM1
RABBITMQ_PORT=5672
RABBITMQ_USER=admin
RABBITMQ_PASSWORD=admin123 # ← debe coincidir con .env.vm1
RABBITMQ_VHOST=/
```

2.2 – Frontend: archivo `.env` o `.env.production`

```
frontend-ocr/.env
VITE_API_URL=http://192.168.18.97:8000 # ← CAMBIAR por la IP
                                         real de VM1
```

Figura 10.2. Puntos de configuración de la IP de la VM1.

El primer archivo a actualizar es `.env.vm2` dentro de la VM2, donde la variable `RABBITMQ_HOST` debe apuntar a la IP real de la VM1 (en el entorno de referencia, 192.168.18.97), manteniendo coincidencia entre `RABBITMQ_PASSWORD` y el valor configurado en `.env.vm1`. El segundo punto de configuración es el archivo `.env` (o `.env.production`) del frontend, donde la variable `VITE_API_URL` debe apuntar también a la IP de la VM1 seguida del puerto 8000, ya que todas las peticiones del frontend viajan siempre hacia el `api_gateway` alojado en la VM1, sin importar en qué máquina se ejecute el propio frontend.

10.3 Checklist de Arranque Completo

Una vez cumplidos los requisitos previos y la configuración de red, el arranque del sistema sigue una secuencia estricta de cinco pasos, ya que cada componente depende de que el anterior esté disponible.

Checklist de arranque completo

1	VM1	<code>> docker compose -f docker-compose.vm1.yml up -d</code> Esperar ~30s a que postgres y rabbitmq esten healthy	 Healthy
2	VM1	<code>> curl http://localhost:8000/health</code> Confirmar status: "ok"	 Status: OK
3	VM2	<code>> docker compose -f docker-compose.vm2.yml up -d</code> (requiere VM1 ya levantada y accesible)	
4	VM2	<code>> docker logs ms_preprocesamiento_vm2</code> Confirmar "Conectado a RabbitMQ en <IP_VM1>:5672"	 Conectado
5	Frontend	<code>npm run build && npx serve dist</code> O abrir directamente con <code>npm run dev</code>	Build Serve Dev 

Figura 10.3. Checklist de arranque completo del sistema.

- En la VM1, ejecutar `docker compose -f docker-compose.vm1.yml up -d` y esperar aproximadamente 30 segundos hasta que tanto PostgreSQL como RabbitMQ reporten el estado "healthy".
- Verificar que el `api_gateway` responde correctamente ejecutando `curl http://localhost:8000/health` y confirmando que el estado devuelto sea "ok".
- En la VM2, ejecutar `docker compose -f docker-compose.vm2.yml up -d`, lo cual requiere que la VM1 ya esté levantada y sea accesible en la red.
- Confirmar, revisando los logs con `docker logs ms_preprocesamiento_vm2`, que el mensaje de log indique "Conectado a RabbitMQ en <IP_VM1>:5672", evidencia de que la comunicación entre VMs quedó correctamente establecida.
- Finalmente, compilar y servir el frontend con `npm run build && npx serve dist`, o alternativamente ejecutarlo en modo desarrollo con `npm run dev`.

10.4 Configuración del Frontend

El frontend puede ejecutarse indistintamente en la VM1, en la VM2 o en una tercera máquina, incluso en el propio navegador del docente, ya que su único requisito de red es poder alcanzar el puerto 8000 de la VM1.

Configuración del Frontend{

El frontend puede correr en VM1, VM2, o en una tercera máquina (incluso el navegador del docente). Solo necesita poder alcanzar el puerto 8000 de VM1.

Variables de entorno

```
cd frontend-ocr
nano .env
.env (producción) - apunta SIEMPRE a VM1, sin importar dónde corra el frontend
VITE_API_URL=http://192.168.18.97:8000 # ← CAMBIAR por la IP real de VM1
```

Compilar y servir

```
npm install
npm run build
npx serve dist -p 5173
O en modo desarrollo con proxy (evita problemas de CORS):
vite.config.js ya tiene el proxy configurado apuntando a 192.168.18.97
Si cambias la IP de VM1, edita también vite.config.js:
target: 'http://NUEVA_IP_VM1:8000'
npm run dev
```

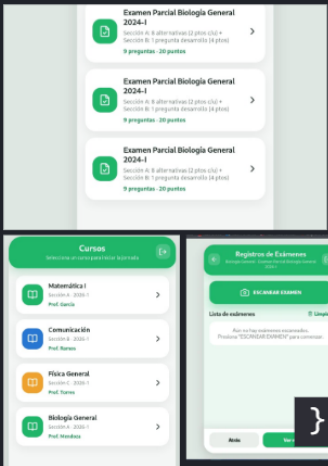


Figura 10.4. Configuración y compilación del frontend.

La configuración se realiza editando el archivo `.env` dentro de la carpeta `frontend-ocr`, fijando la variable `VITE_API_URL` con la IP real de la VM1 y el puerto 8000; esta variable debe apuntar siempre a la VM1, independientemente de dónde se ejecute el frontend. Para producción, el flujo es `npm install`, `npm run build` y `npx serve dist -p 5173`. Para desarrollo local con proxy (evitando así problemas de CORS), el archivo `vite.config.js` ya contiene un proxy preconfigurado apuntando a la IP de referencia (192.168.18.97); si dicha IP cambia, debe actualizarse tanto en `vite.config.js` como en el `.env` antes de ejecutar `npm run dev`.

11. Integración y Entrega Continua (CI/CD) con Jenkins

Para garantizar que los cambios de código de los cuatro integrantes del equipo no rompan el sistema al integrarse, se implementó un pipeline de integración continua con Jenkins, distribuido igualmente entre las dos máquinas virtuales del proyecto, aprovechando la arquitectura de nodo maestro (master) y nodo agente (agent) propia de Jenkins.

11.1 Distribución del Trabajo entre VM1 y VM2

Así se Distribuye el Trabajo: jenkins + testcontainers

VM1 – Jenkins Master (192.168.18.97)	
└─ Contenedor Jenkins corriendo	
└─ Stage 1: Checkout	→ corre en VM1
└─ Stage 2: Tests VM1	→ corre en VM1 (Testcontainers: Postgres + RabbitMQ)
└─ Stage 4: Deploy VM1	→ corre en VM1 (docker compose vm1)
└─ Stage 6: Tests E2E	→ corre en VM1 (HTTP a localhost:8000)
VM2 – Jenkins Agent	
└─ Java 17 instalado (requisito del agente)	
└─ Stage 3: Tests VM2	→ corre en VM2 (Testcontainers: RabbitMQ + Tesseract real)
└─ Stage 5: Deploy VM2	→ corre en VM2 (docker compose vm2)
Jenkins Master (VM1) —SSH→ Jenkins Agent (VM2)	
Asigna stages	Ejecuta y reporta resultados

Figura 11.1. Distribución de etapas del pipeline entre Jenkins Master y Agent.

La VM1 aloja el contenedor de Jenkins Master y ejecuta directamente las etapas que dependen de su propio entorno: el checkout del código fuente (Stage 1), las pruebas de los servicios propios de la VM1 usando Testcontainers para levantar instancias efímeras de PostgreSQL y RabbitMQ (Stage 2), el despliegue de los servicios de la VM1 mediante docker compose (Stage 4) y, al final del pipeline, las pruebas end-to-end (E2E) contra `http://localhost:8000` (Stage 6).

La VM2, por su parte, se configura como Jenkins Agent —requiriendo Java 17 instalado como prerequisite— y ejecuta las etapas correspondientes a su propio entorno: las pruebas de los servicios de la VM2 (Stage 3), utilizando Testcontainers para RabbitMQ y una instancia real de Tesseract, y el despliegue de los servicios de la VM2 (Stage 5). La comunicación entre el maestro y el agente se realiza mediante SSH, donde el maestro asigna las etapas correspondientes y el agente las ejecuta y reporta sus resultados de vuelta.

Esta distribución asegura que las pruebas que requieren Tesseract real (más pesadas en cómputo) se ejecuten en la misma máquina donde Tesseract se despliega en producción, evitando falsos positivos que podrían ocurrir si se probaran en un entorno distinto al de despliegue final.

11.2 Preparación de la VM1 (Jenkins Master)

Figura 11.3. Configuración de la VM2 como agente de Jenkins.

En la VM2 se ejecuta el script `setup-agent-vm2.sh`, al cual se le pasa como parámetro la clave pública SSH del maestro (generada previamente en la VM1), de modo que Jenkins Master pueda conectarse por SSH sin intervención manual. El script instala las dependencias del sistema necesarias para el agente y deja preparados los datos requeridos para registrar el nodo desde el panel web de Jenkins: la IP de la VM2 (192.168.18.96 en el entorno de referencia), el usuario SSH (vm1), el directorio raíz del agente (/home/vm1/jenkins-agent), la etiqueta del agente (vm2-agent), el entorno virtual de Python para pruebas (~/.venv-jenkins) y el script wrapper de pytest (~/.run-pytest-vm2.sh).

11.4 Configuración Inicial de Jenkins en el Navegador

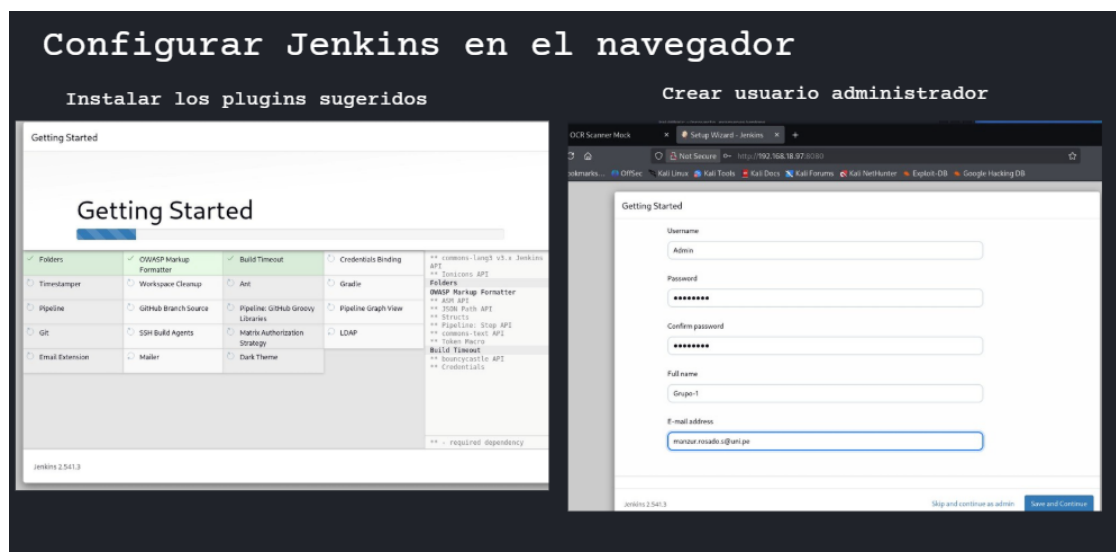


Figura 11.4. Asistente de configuración inicial de Jenkins.

Al acceder por primera vez a Jenkins a través del navegador (<http://192.168.18.97:8080> en el entorno de referencia), el asistente de configuración inicial guía la instalación de los plugins sugeridos —entre ellos Pipeline, Git, SSH Build Agents, Credentials Binding y Timestamp, indispensables para el tipo de pipeline distribuido descrito en esta sección— y la creación del usuario administrador del equipo, con su nombre completo y correo institucional.

11.5 Verificación de Nodos Registrados

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-in Node	Linux (amd64)	In sync	49.30 GiB	533.86 MiB	49.30 GiB	0ms
	vm2-agent	Linux (amd64)	4.6 sec ahead	2.02 GiB	2.56 GiB	1.99 GiB	45ms
	Data obtained	1 min 58 sec	1 min 57 sec	1 min 57 sec	1 min 57 sec	1 min 57 sec	1 min 58 sec

Figura 11.5. Panel de nodos de Jenkins con el agente VM2 registrado.

Una vez completada la configuración, el panel de nodos de Jenkins (Manage Jenkins → Nodes) permite verificar que, además del nodo integrado (Built-In Node) correspondiente a la propia VM1, el nodo vm2-agent aparece registrado y en línea, con su arquitectura (Linux amd64), diferencia de reloj, espacio en disco y memoria disponibles. Esta pantalla constituye la verificación final de que el pipeline distribuido puede efectivamente asignar y ejecutar las etapas 3 y 5 en la VM2, tal como se definió en la sección 11.1.

12. Conclusiones

- El análisis comparativo entre los procesos AS-IS y TO-BE demuestra que el sistema no busca reemplazar el criterio del docente, sino eliminar el trabajo repetitivo de transcripción y búsqueda manual, dejando la intervención humana reservada únicamente a los casos de baja confianza de lectura.
- La decisión de mantener el examen en papel, en lugar de migrar a una plataforma 100% digital, responde directamente a la realidad de instituciones educativas sin acceso garantizado a un dispositivo por alumno, lo que amplía significativamente la viabilidad de adopción del sistema frente a soluciones puramente digitales.
- La separación de la arquitectura en dos máquinas virtuales, con el procesamiento de imagen aislado en la VM2, permite que el componente más intensivo en cómputo (OCR y visión por computadora) escale o se reemplace de forma independiente sin afectar la disponibilidad del resto del sistema.
- El uso de RabbitMQ como middleware de mensajería entre ambas VMs otorga tolerancia a fallos temporales: si el servidor de OCR se desconecta, los exámenes pendientes permanecen en cola y se procesan automáticamente en cuanto la conexión se restablece.
- La incorporación de un pipeline de Jenkins con nodo maestro y agente distribuido entre ambas VMs permite validar automáticamente tanto la lógica de negocio como el procesamiento real con Tesseract antes de cada despliegue, reduciendo el riesgo de introducir errores al integrar el trabajo de los cuatro integrantes del equipo.